

Understanding Model Context Protocols (MCPs)

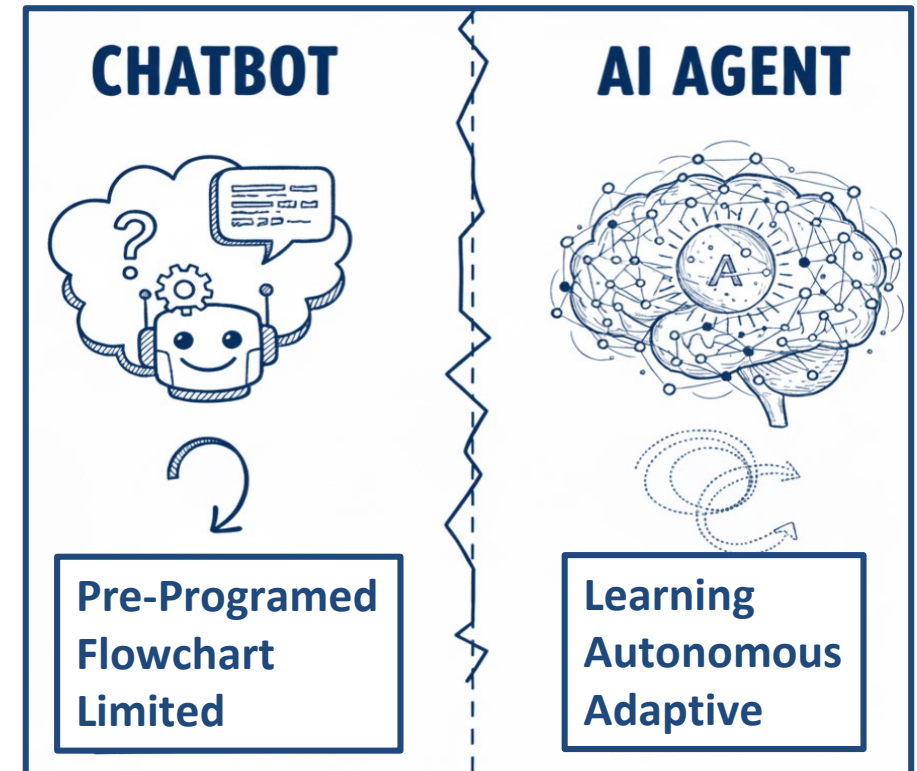
The Agency Problem

Chatbots vs. Agents

What's the difference between a chatbot that can answer your questions and an autonomous AI agent that can truly do things for you?

Traditional LLMs are fantastic at generating text but have no inherent ability to interact with the real world. They can't access live data, run software, or execute meaningful tasks.

This isolation creates a fundamental barrier between AI understanding and AI action—the core challenge we call the "agency problem."



Introducing Model Context Protocol

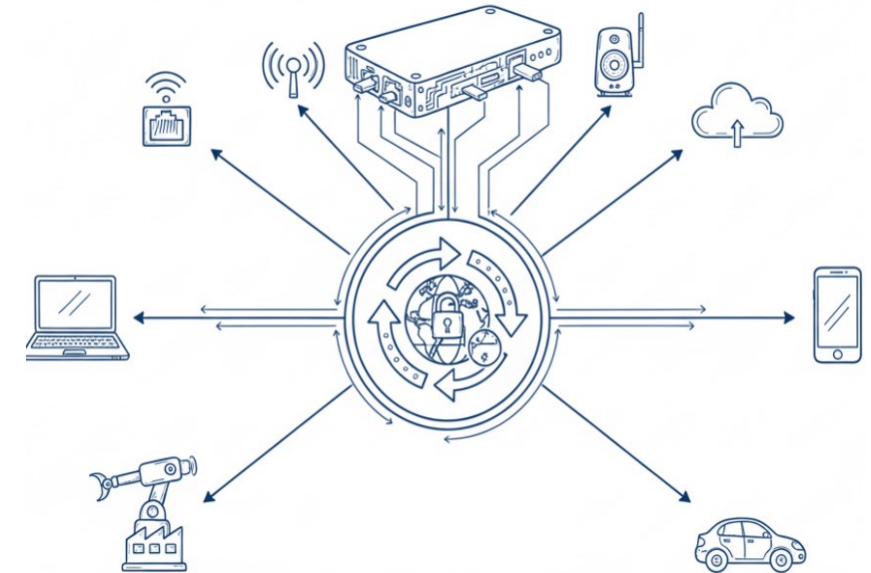
- Model Context Protocol (MCP) is a transformative solution—an open standard that provides a consistent, predictable way for LLMs to interface with external resources.

Universal Connection

Like a USB-C port for AI systems, MCP creates standardized connections between language models and external tools.

Real-World Action

Transforms passive text generators into active agents capable of booking flights, managing projects, and executing complex workflows.



What Are Model Context Protocols?

Structured Methodology

A set of rules and conventions for managing information sent to AI models

Expert Briefing

Like providing a comprehensive dossier to an expert before they tackle a task

Architectural Blueprint

Transforms forgetful LLMs into coherent, consistent AI partners

The Three Pillars of MCP



Resources

Static data sets that agents can read from—documents, calendars, database records, user preferences, and configuration files. These provide the contextual foundation for intelligent decision-making.



Tools

Enabling models to interact with external systems and perform actionable functions with clear definitions. These are the hands that allow agents to manipulate the external world.



Prompts

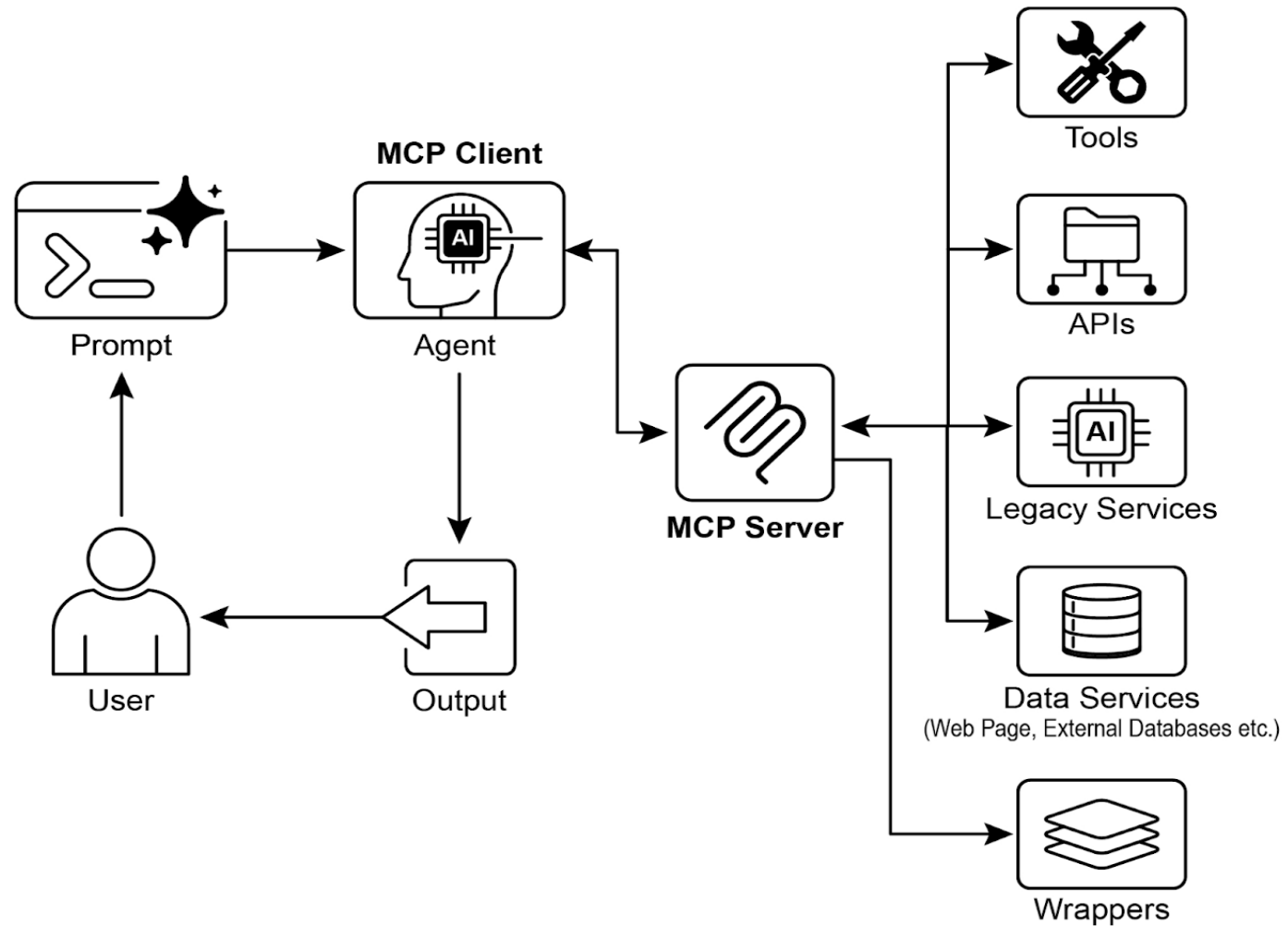
Structured templates and guidance that ensure agents interact with tools correctly. They provide formatting rules and response handling patterns for consistent behavior.

MCP vs. Traditional Tool Calling

Feature	Tool Function Calling	Model Context Protocol (MCP)
Standardization	Proprietary and vendor-specific. The format and implementation differ across LLM providers.	An open, standardized protocol, promoting interoperability between different LLMs and tools.
Scope	A direct mechanism for an LLM to request the execution of a specific, predefined function.	A broader framework for how LLMs and external tools discover and communicate with each other.
Architecture	A one-to-one interaction between the LLM and the application's tool-handling logic.	A client-server architecture where LLM-powered applications (clients) can connect to and utilize various MCP servers (tools).
Discovery	The LLM is explicitly told which tools are available within the context of a specific conversation.	Enables dynamic discovery of available tools. An MCP client can query a server to see what capabilities it offers.
Reusability	Tool integrations are often tightly coupled with the specific application and LLM being used.	Promotes the development of reusable, standalone "MCP servers" that can be accessed by any compliant application.

[Source: Agentic Design Patterns](#)

Envisioned MCP Architecture



Agentic Workflow: Step-by-Step

Let's walk through a real scenario:

"Find all Q4 sales figures for the Southeast region and email them to Jane."

01

User Request Processing

Agent receives natural language request and begins internal reasoning using Chain-of-Thought to break down the task requirements.

03

Data Retrieval

Agent invokes `get_sales_data` tool with parameters "Q4" and "Southeast", server translates to database query and returns structured JSON.

02

Tool Discovery

MCP client queries available servers, finding "SalesDatabase" server with `get_sales_data` tool and "EmailService" server with `send_email` capability.

04

Action Execution

Agent processes returned data, creates summary, then invokes `send_email` tool with recipient, subject, and formatted content.

Key Influencers of MCP Suitability

- **Components:** MCP standardizes three elements:
 - **Resources** (static data like files)
 - **Tools** (executable functions)
 - **Prompts** (interaction templates)
- **Discoverability:** Agents can dynamically query servers to "discover" new tools and resources, allowing them to adapt to new capabilities on the fly.
- **Security:** Robust authentication and authorization are essential to control which clients can access servers and what actions they can perform.
- **Implementation:** SDKs from providers like Anthropic are simplifying development by abstracting away boilerplate code.
- **Error Handling:** The protocol defines how errors are communicated back to the LLM, enabling the agent to handle failures and try alternative solutions.
- **Deployment:** Servers can be local (for security and speed) or remote (for shared access).

Benefits of Protocol-Driven Approach

Reliability

Consistent, predictable outputs
essential for business
applications

Enhanced Capabilities

Tool integration transforms LLMs
into active agents

Improved Accuracy

RAG integration grounds models
in factual, real-time data

Simplified Debugging

Structured logs provide clear
audit trails

Accelerated Development

Reusable blueprints for context
management

Key Enabled Capabilities

Real-Time Intelligence

Agents work with live, current data rather than pre-trained knowledge, enabling responses to dynamic conditions and up-to-date information.

Modular Architecture

Standardized protocol means swapping servers doesn't require agent logic changes - maintain consistency while enabling flexibility.

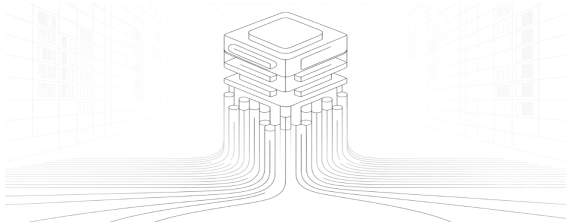
Scalable Integration

Single implementation works across unlimited external services, dramatically reducing development overhead and maintenance complexity.

Real World Applications of MCPs

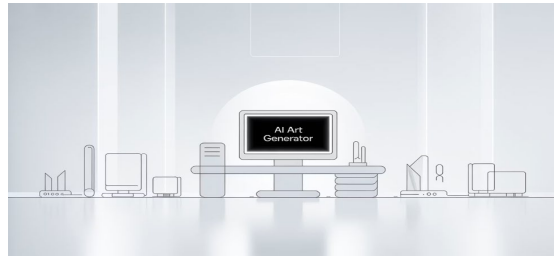
Enterprise Integration

Connect agents to CRM systems, ERP databases, and internal APIs for automated customer service, sales reporting, and operational workflows.



Generative Media

Command services like Imagen and Veo to create images and videos from natural language prompts, enabling creative automation at scale.



Financial Services

Automate complex trading workflows by connecting to live market data, risk assessment tools, and trading platforms for intelligent financial decisions.



IoT and Physical World Control

Smart Environment Management

- MCP enables agents to control physical devices in smart homes and industrial facilities, bridging digital intelligence with real-world automation.
 - Home automation systems and IoT devices
 - Industrial equipment and factory controls
 - Environmental monitoring and response
 - Security systems and access management

This creates seamless integration between AI decision-making and physical world actions, enabling truly responsive environments.



The Future of MCPs

MCPs are a critical step toward creating interconnected AI ecosystems.



Industry Standardization

Emergence of HTTP-like standards for model context, enabling greater interoperability



Multi-Agent Systems

Teams of specialized agents collaborating through sophisticated cross-system protocols



Efficient Context Management

Advanced techniques for compressing and prioritizing information across extended timeframes



On-Device Integration

Lightweight, efficient protocols adapted for local model constraints

Enabling True AI Agency



Bridging the Gap

MCP transforms isolated language models into connected, capable agents that can interact with the real world.



Universal Standard

Open protocol enables consistent, predictable integrations across unlimited external systems and services.



Future Ready

Foundation for multi-agent ecosystems and truly autonomous AI systems that can collaborate and execute complex workflows.

Model Context Protocol is the essential bridge between AI understanding and AI action—turning language models into true agents capable of meaningful real-world impact.

Key Takeaways

- The Model Context Protocol (MCP) is an open standard facilitating standardized communication between LLMs and external applications, data sources, and tools.
- It employs a client-server architecture, defining the methods for exposing and consuming resources, prompts, and tools.
- Tools are currently in development that can simplify the development and management of MCP servers, particularly for exposing tools implemented in Python.
- MCP enables LLMs and agents to interact with real-world systems, access dynamic information, and perform actions beyond text generation.